

Extração de textos digitalizados com PHP e OCR (Artigo: Isaac Caraça)

Autor: Isaac Caraça

Problema Real:

Durante o desenvolvimento do novo sistema de leis municipais de Guarulhos, foi identificado um problema na conversão de arquivos PDF para texto. Parte significativa dos documentos, especialmente os do tipo 2 – Projeto de Lei, apresentava conteúdo ilegível após o processamento com o PDF Parser, impossibilitando a indexação e a pesquisa textual.

Esses arquivos, por serem PDFs escaneados (imagens digitalizadas e não texto real), retornavam caracteres corrompidos e ilegíveis, comprometendo a utilidade do campo `conteudo_pdf` no banco de dados.

No processamento original, utilizava-se a biblioteca PDF Parser para converter os arquivos em texto e armazenar o resultado na coluna `conteudo_pdf`. Esse método funcionava corretamente para PDFs que continham texto embutido, mas falhava com arquivos escaneados.

Um exemplo do conteúdo extraído de um desses arquivos é o seguinte trecho corrompido:

```
352-(72' (/ (,1  
'LVSROGR VREUH 3,QVWLWXL QRUPDV JHUDLV SDUD IXQFLRQDPHQWR GH FDL[DV  
UHJLVWUDGRUDVQRkPELWRGR0XQLFtSLRHGiRXWUDVSURYLGrQFLDV'
```

...

Esse tipo de resultado tornava inviável a pesquisa textual, uma vez que o sistema deveria permitir buscas tanto pela ementa quanto pelo conteúdo completo do documento.

Diante do problema, optou-se por aplicar Reconhecimento Óptico de Caracteres (OCR) aos PDFs escaneados, utilizando o Tesseract OCR, uma ferramenta de código aberto capaz de reconhecer texto em imagens.

O processo envolveu a conversão de cada página do PDF em imagem e, posteriormente, o reconhecimento de texto em português.

Antes da automatização, foram realizados testes manuais pelo Prompt de Comando (CMD), para verificar a precisão do OCR e validar a extração correta do conteúdo textual.

Após os testes bem-sucedidos, o processo foi automatizado através da criação do método:

Solução para o Problema:

Método:

```
public function atualizarConteudoPdfOCRLote(int $inicioId, int
$ tamanhoLote = 100)
{
    require 'vendor/autoload.php';
    $conexao = \sistema\Nucleo\Conexao::getInstancia();

    $popplerPath =
"C:\Poppler\Release-23.10.0-0\poppler-23.10.0\Library\bin\pdftop
pm.exe";
    $tesseractPath = "C:\Program
Files\Tesseract-OCR\tesseract.exe";
    $tmpDir = sys_get_temp_dir();

    // Buscar URLs base
    $leiCaminhos = $conexao->query("SELECT * FROM
leis.lei_caminho")->fetchAll(\PDO::FETCH_ASSOC);
    $urlsPorTipo = [];
    foreach ($leiCaminhos as $c) {
        $urlsPorTipo[$c['tipo_caminho']] = rtrim($c['caminho'],
'/');
    }
    $raizLeis = $urlsPorTipo[6] ??
'https://www.guarulhos.sp.gov.br/06_prefeitura/leis/';
```

```

        // Buscar lote de leis
        $leis = $conexao->prepare("
            SELECT id_lei, arquivo_lei, tipo
            FROM leis.lei
            WHERE id_lei >= :id
            ORDER BY id_lei ASC
            LIMIT $tamanhoLote
        ");
        $leis->execute([':id' => $inicioId]);
        $leis = $leis->fetchAll(\PDO::FETCH_ASSOC);

        if (!$leis) {
            echo "⚠ Nenhum registro encontrado a partir do ID $inicioId\n";
            return;
        }

        echo "🚀 Processando lote a partir do ID $inicioId com " .
count($leis) . " registros...\n";

        $downloads = [];
        foreach ($leis as $lei) {
            $arquivoRaw = $lei['arquivo_lei'] ?? '';
            $arquivoLimpo = ltrim(preg_replace('#^\.\./#', '',
trim($arquivoRaw)), '/');

            // ✅ Casos especiais
            $basePath = $urlsPorTipo[$lei['tipo']] ?? $raizLeis;

            // Decretos
            if (preg_match('/^\.decretos_\d{4}\\/', $arquivoRaw,
$matches)) {
                $ano = $matches[1];
                $basePath =
"https://www.guarulhos.sp.gov.br/06_prefeitura/leis/decretos_$ano/";
                $arquivoLimpo = preg_replace('/^\.decretos_\d{4}\\/',
'', $arquivoLimpo);
            }

            // Autógrafos
            elseif ((int)$lei['tipo'] === 4) {
                if
(preg_match('/^autografos\d{3}_\d{4}_autografo\.pdf$/',
$arquivoLimpo, $m)) {
                    $arquivoLimpo = $m[1];
                }
                $basePath = $urlsPorTipo[4] ?? $raizLeis;
            }

            // Para leis, portarias, projetos: manter padrão pelo tipo
            else {
                $basePath = $urlsPorTipo[$lei['tipo']] ?? $raizLeis;
            }
        }
    }
}

```

```

    }

    $urlPdf = rtrim($basePath, '/') . '/' .
rawurlencode($arquivoLimpo);

    if (empty($arquivoLimpo) || !$basePath) {
        echo "⚠ Lei {$lei['id_lei']} sem caminho válido\n";
        continue;
    }

    $downloads[] = [
        'id' => $lei['id_lei'],
        'url' => $urlPdf,
        'dest' => "$tmpDir/lei_{$lei['id_lei']}.pdf"
    ];
}

// Baixar PDFs em paralelo
$mh = curl_multi_init();
$chList = [];

foreach ($downloads as $d) {
    $ch = curl_init($d['url']);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true);
    curl_setopt($ch, CURLOPT_TIMEOUT, 120);

    $chList[$d['id']] = ['ch' => $ch, 'dest' => $d['dest'],
'url' => $d['url']];
    curl_multi_add_handle($mh, $ch);
}

$running = null;
do {
    curl_multi_exec($mh, $running);
    curl_multi_select($mh);
} while ($running > 0);

foreach ($chList as $id => $info) {
    $conteudo = curl_multi_getcontent($info['ch']);
    $httpCode = curl_getinfo($info['ch'], CURLINFO_HTTP_CODE);
    $curlError = curl_error($info['ch']);

    if ($conteudo && $httpCode === 200) {
        file_put_contents($info['dest'], $conteudo);
        echo "📄 Lei $id baixada com sucesso | PDF:
{$info['dest']} | HTTP: $httpCode\n";
    } else {
        echo "❌ Falha no download da lei $id | URL:
{$info['url']} | HTTP: $httpCode | Erro cURL: $curlError\n";
    }
}

```

```

        curl_multi_remove_handle($mh, $info['ch']);
        curl_close($info['ch']);
    }
    curl_multi_close($mh);

    foreach ($downloads as $d) {
        $id = $d['id'];
        $pdf = $d['dest'];
        $baseName = "$tmpDir/lei_$id";

        if (!file_exists($pdf)) {
            echo "⚠️ PDF da lei $id não encontrado, pulando...\n";
            continue;
        }

        echo "🔍 Convertendo lei $id em imagens...\n";
        $cmdPdfToPng = "$popplerPath -png -r 150 \"$pdf\" \"$baseName\" 2>nul";
        exec($cmdPdfToPng);

        $paginas = glob("$baseName-*.png");
        if (!$paginas) {
            echo "❌ Nenhuma página encontrada no PDF da lei $id\n";
            continue;
        }

        foreach ($paginas as $pagina) {
            $saida = str_replace('.png', '', $pagina);
            exec("$tesseractPath \"$pagina\" \"$saida\" -l por");
        }

        $textoCompleto = '';
        foreach (glob("$tmpDir/lei_$id-*.txt") as $txtPath) {
            if (file_exists($txtPath)) {
                $textoCompleto .= file_get_contents($txtPath) . "\n\n";
            }
            @unlink($txtPath);
        }

        if (!empty(trim($textoCompleto))) {
            $update = $conexao->prepare("UPDATE leis.lei SET conteudo_pdf = :texto WHERE id_lei = :id");
            $update->execute([':texto' => $textoCompleto, ':id' => $id]);
            echo "✅ Lei $id atualizada com sucesso\n";
        } else {
            echo "⚠️ Nenhum texto reconhecido na lei $id\n";
        }
    }
}

```

```

        @unlink("$tmpDir/lei_$id.pdf");
        foreach (glob("$tmpDir/lei_$id-*.png") as $png) {
            @unlink($png);
        }
    }

    echo "🏁 Lote finalizado!\n";

}

```

Explicação:

```

public function atualizarConteudoPdfOCRLote(int $inicioId, int
$tamanhoLote = 100)

```

Função: processa um lote de leis a partir de um ID inicial.

Parâmetros:

\$inicioId:

o primeiro ID a ser processado;

\$tamanhoLote:

número máximo de registros por execução.

```

require 'vendor/autoload.php';
$conexao = \sistema\Nucleo\Conexao::getInstancia();

$popplerPath =
"C:\\Poppler\\Release-23.10.0-0\\poppler-23.10.0\\Library\\bin\\pdftop
pm.exe";
$tesseractPath = "C:\\Program Files\\Tesseract-OCR\\tesseract.exe";

$tmpDir = sys_get_temp_dir();

```

Carrega o autoload do Composer, garantindo o acesso às classes do projeto;

Obtém a instância do banco via PDO;

Define os caminhos locais das ferramentas Poppler (para converter PDF → PNG) e Tesseract (para OCR).

sys_get_temp_dir():

cria um diretório temporário do sistema, evitando poluir o projeto com arquivos intermediários.

```
$leiCaminhos = $conexao->query("SELECT * FROM
leis.lei_caminho")->fetchAll(\PDO::FETCH_ASSOC);
$urlsPorTipo = [];
foreach ($leiCaminhos as $c) {
    $urlsPorTipo[$c['tipo_caminho']] = rtrim($c['caminho'], '/');
}
}
```

Cada tipo de lei (Decreto, Portaria, Projeto, etc.) tem um caminho base distinto no site.

O sistema busca esses caminhos no banco para formar a URL de download correta.

```
$leis = $conexao->prepare("
    SELECT id lei, arquivo lei, tipo
    FROM leis.lei
    WHERE id lei >= :id
    ORDER BY id lei ASC
    LIMIT $tamanhoLote
");
```

Seleciona um lote de leis a partir do ID informado.

O limite impede sobrecarga na memória, facilitando o processamento de grandes volumes de documentos.

```
$downloads[] = [
    'id' => $lei['id_lei'],
    'url' => $urlPdf,
    'dest' => "$tmpDir/lei_{$lei['id_lei']}.pdf"
];
```

Monta array downloads com id, url e dest (destino temporário).

```
$mh = curl_multi_init();
$chList = [];

foreach ($downloads as $d) {
    $ch = curl_init($d['url']);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    curl_setopt($ch, CURLOPT_FOLLOWLOCATION, true);
    curl_setopt($ch, CURLOPT_TIMEOUT, 120);

    $chList[$d['id']] = ['ch' => $ch, 'dest' => $d['dest'], 'url' =>
$d['url']];
    curl_multi_add_handle($mh, $ch);
}

$running = null;
```

```
do {
    curl_multi_exec($mh, $running);
    curl_multi_select($mh);

} while ($running > 0);
```

Inicializa o multi-handle `curl_multi_init()` para downloads paralelos.

Para cada download, cria um curl normal, define opções e o adiciona ao multi.

Executa o loop `curl_multi_exec` + `curl_multi_select` até a conclusão (quando `$running` fica 0).

CURLOPT_RETURNTRANSFER:

retorna o corpo como string via `curl_multi_getcontent`.

CURLOPT_FOLLOWLOCATION:

segue redirecionamentos HTTP (útil se a URL faz 302 → localização final).

CURLOPT_TIMEOUT:

timeout em segundos para a operação.

```
foreach ($chList as $id => $info) {
    $conteudo = curl_multi_getcontent($info['ch']);
    $httpCode = curl_getinfo($info['ch'], CURLINFO_HTTP_CODE);
    $curlError = curl_error($info['ch']);

    if ($conteudo && $httpCode === 200) {
        file_put_contents($info['dest'], $conteudo);
        echo "📄 Lei $id baixada com sucesso | PDF: {$info['dest']} | HTTP: $httpCode\n";
    } else {
        echo "❌ Falha no download da lei $id | URL: {$info['url']} | HTTP: $httpCode | Erro cURL: $curlError\n";
    }

    curl_multi_remove_handle($mh, $info['ch']);
    curl_close($info['ch']);
}

curl_multi_close($mh);
```

Para cada handle, obtém conteúdo e código HTTP.

Se HTTP 200 e `$conteudo` não vazio, grava no destino (`file_put_contents`).

Remove e fecha cada handle; fecha o multi-handle ao final.

```
foreach ($downloads as $d) {
    $id = $d['id'];
    $pdf = $d['dest'];
```



```

$baseName = "$tmpDir/lei_$id";

if (!file_exists($pdf)) {
    echo "⚠ PDF da lei $id não encontrado, pulando...\n";
    continue;
}

echo "🔄 Convertendo lei $id em imagens...\n";
$cmdPdfToPng = "$popplerPath -png -r 150 \"$pdf\" \"$baseName\"
2>nul";
exec($cmdPdfToPng);

$paginas = glob("$baseName-*.png");
if (!$paginas) {
    echo "❌ Nenhuma página encontrada no PDF da lei $id\n";
    continue;
}

foreach ($paginas as $pagina) {
    $saida = str_replace('.png', '', $pagina);
    exec("$tesseractPath \"$pagina\" \"$saida\" -l por");
}

}

```

Para cada PDF baixado:

Checa se o arquivo existe.

Constrói comando pdftoppm para exportar páginas como baseName-1.png, baseName-2.png, etc. (-r 150 define 150 DPI).

Usa glob() para coletar as imagens geradas.

Para cada imagem, executa tesseract imagem saida -l por — gerando saida.txt.

pdftoppm -png -r 150 → -png cria PNG; -r 150 é resolução balanceada (150 DPI é um bom trade-off entre precisão e tamanho).

2>nul (Windows) redireciona stderr para nul — suprime mensagens. Em Linux seria 2>/dev/null.

glob("\$baseName-*.png") retorna array de arquivos gerados.

```

$textoCompleto = '';
foreach (glob("$tmpDir/lei_$id-*.txt") as $txtPath) {
    if (file_exists($txtPath)) {
        $textoCompleto .= file_get_contents($txtPath) . "\n\n";
        @unlink($txtPath);
    }
}

}

```

Lê todos os arquivos .txt gerados pelo Tesseract (lei_123-1.txt, etc.), concatena em \$textoCompleto separando por duas quebras de linha.

Em seguida tenta remover (unlink) cada .txt (o @ suprime warnings se falhar).

Cria um único bloco de texto pronto para armazenar no banco.

```
if (!empty(trim($textoCompleto))) {  
    $update = $conexao->prepare("UPDATE leis.lei SET conteudo_pdf =  
:texto WHERE id lei = :id");  
    $update->execute([':texto' => $textoCompleto, ':id' => $id]);  
    echo "✅ Lei $id atualizada com sucesso\n";  
} else {  
    echo "⚠️ Nenhum texto reconhecido na lei $id\n";  
}
```

Se houver texto (após trim) atualiza a coluna conteudo_pdf para o id_lei correspondente.

Usa prepared statement para evitar SQL injection e problemas com aspas.

Salva o texto reconhecido no banco, tornando-o pesquisável.

```
@unlink("$tmpDir/lei $id.pdf");  
foreach (glob("$tmpDir/lei_$id-*.png") as $png) {  
    @unlink($png);  
}
```

Remove o PDF temporário e as imagens PNG geradas por página.

Processamento:

```
<?php  
  
require_once 'sistema/Modelo/BuscaLei.class.php';  
use sistema\Modelo\BuscaLei;  
  
set_time_limit(0);  
ini_set('memory_limit', '2G');  
  
$inicio = 21312;  
$limitePorLote = 1;  
$maxId = 21313;  
  
while ($inicio < $maxId) {  
    echo "\n ♦ Processando lote a partir do ID $inicio...\n";  
  
    try {  
        $objLei = new BuscaLei();  

```

```

        $objLei->atualizarConteudoPdfOCRLote($inicio,
$limitePorLote);

    } catch (\PDOException $e) {
        echo "❌ Erro de conexão, tentando reconectar: " .
$e->getMessage() . "\n";
        sleep(5);
        continue;
    }

    $inicio += $limitePorLote;

    gc_collect_cycles();
}

echo "\n🏁 Todos os lotes processados!\n";

?>

```

Explicação:

```
require_once '...BuscaLei.class.php';
```

carrega a classe que contém atualizarConteudoPdfOCRLote.

```
use sistema\Modelo\BuscaLei;
```

importa o namespace.

```
set_time_limit(0);
```

remove limite de tempo de execução (útil para scripts longos).

```
ini_set('memory_limit', '2G');
```

aumenta limite de memória (ajuste conforme necessidade/servidor).

```
$inicio, $limitePorLote, $maxId
```

controla intervalo de IDs a processar.

```
while ($inicio < $maxId)
```

loop executando lotes de IDs até atingir maxId.

```
try { ... } catch (\PDOException $e) { ... }
```

tenta executar o método e captura exceções de banco (reconecta após sleep(5)).

```
gc_collect_cycles();
```

força coleta de ciclos para liberar memória em longas execuções.

Controla a execução gradual (pode ser rodada no CRON/Task Scheduler).

Permite reiniciar a partir de início caso seja interrompido.

Na prática, execute:

```
PS C:\xampp\htdocs\leis_externo> php [nome_arquivo].php
```

```
◆ Processando lote a partir do ID 4851...
```

```
📄 Lei 4851 baixada com sucesso | PDF:
```

```
C:\Users\ISAACC~1\AppData\Local\Temp\lei_4851.pdf | HTTP: 200
```

```
🔍 Convertendo lei 4851 em imagens...
```

```
✅ Lei 4851 atualizada com sucesso
```

```
🏁 Lote finalizado!
```

Com isso se gera o resultado:

Resultado:

PROJETO DE LEI N° 001/2001

Dispondo sobre: "Institui normas gerais para funcionamento de caixas registradoras no âmbito do Município e dá outras providências".

A CÂMARA MUNICIPAL DE GUARULHOS APROVA:

Artigo 1º. – Ficam as agências - bancárias, super e hiper-mercados instalados no Município - obrigadas a manterem em funcionamento para atendimento ao consumidor, a proporcionalidade de 100% (cem por cento), de funcionários em operação nos respectivos caixas instalados em cada estabelecimento individual no local.

Parágrafo único – Não serão permitidas às agências bancárias, super e hiper mercados, a retirada de caixas já devidamente instalados conforme planta aprovada pela prefeitura, sob pena de infração prevista no item 4 do artigo 2º da presente lei.

Artigo 2º. – O não cumprimento da presente Lei, sujeitará ao infrator às seguintes penalidades:

1. Notificação;

2. Advertência;

3. Multa de 2.500 "UFG" - Unidade Fiscal de Guarulhos, na primeira reincidência;

4. Multa de 5.000 "UFG" - Unidade Fiscal de Guarulhos, na segunda reincidência;

5. Suspensão temporária por 60 (sessenta) dias do Alvará Municipal de funcionamento, até sua plena regularização.

Parágrafo único - Com a regularização da infração, o Alvará de Funcionamento Municipal suspenso, voltará a sua normalidade, após comunicação por parte da Instituição autuada, a autoridade competente da "SICA" - Secretaria de Indústria, Comércio e Abastecimento - órgão municipal responsável pelo cumprimento da presente Lei.

Artigo 3º. - A "SICA" - Secretaria de Indústria, Comércio e Abastecimento será o órgão municipal competente para fiscalizar e autuar, zelando pelo cumprimento da presente Lei.

Parágrafo único - A "SICA" - Secretaria de Indústria, Comércio e Abastecimento - será também responsável pelos seguintes procedimentos desta Lei, a saber:

1. Receber os eventuais recursos de multas aplicadas pelos Agentes Fiscais, julgando-os e decidindo no prazo de 8 (oito) dias, o auto de infração lavrado e assinado em 2 (duas) vias pelo Infrator e pelo Agente Fiscal;

2. Considerar como aceito, apenas os recursos feitos através de petição, protocolado na própria "SICA" - Secretária de Indústria, Comércio e Abastecimento - o qual não terá efeito suspensivo.

Artigo 4º. - As agências bancárias, os super e hiper-mercados, têm o prazo de 60 (sessenta) dias, a contar da data de publicação desta Lei, para se adaptarem às novas regras estabelecidas.

Artigo 5º. - A exemplo do que já ocorre com o procedimento das agências bancárias, os super e hiper-mercados, também serão obrigados a manter, no mínimo 1

dirigem à esses estabelecimentos, membros de uma coletividade de consumo.

A presente situação não pode mais continuar, tem se transformado em um verdadeiro caos, merecendo, portanto, enérgicas providências do Poder Público.

Sala das Sessões, 19 de janeiro de 2001.

JOÃO DÁRCIO FILHO

VEREADOR